Analityka anomalii w Ataku DDoS

Wykrywanie ataku DDoS metodą bootstrap – raport z kodem w Pythonie

Opis scenariusza

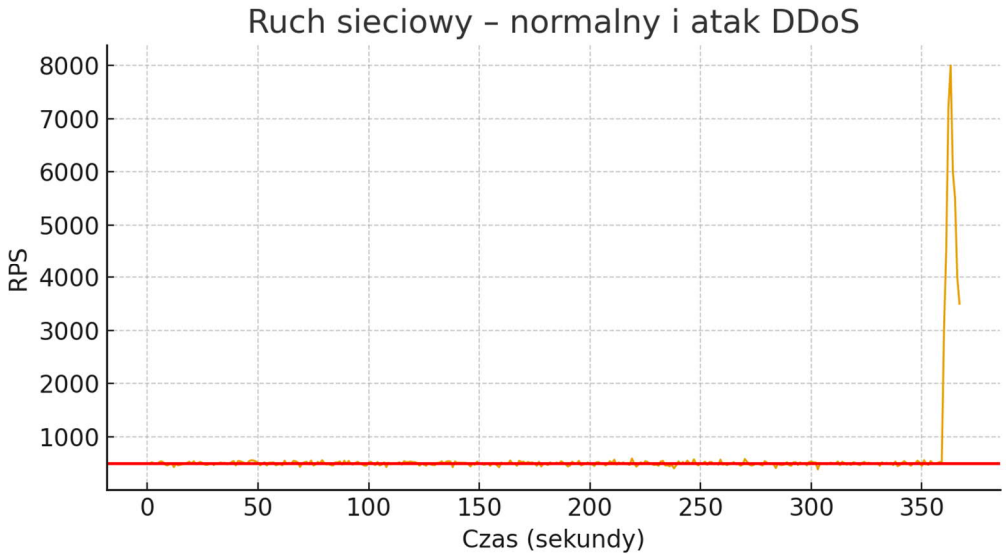
W przykładzie analizujemy liczbę zapytań HTTP na sekundę (RPS) przychodzących do serwera. Dane historyczne zawierają typowe wartości ruchu (~500 RPS), natomiast w ostatnich 8 sekundach występuje gwałtowny wzrost wskazujący na potencjalny atak DDoS.

Bootstrapowy przedział ufności

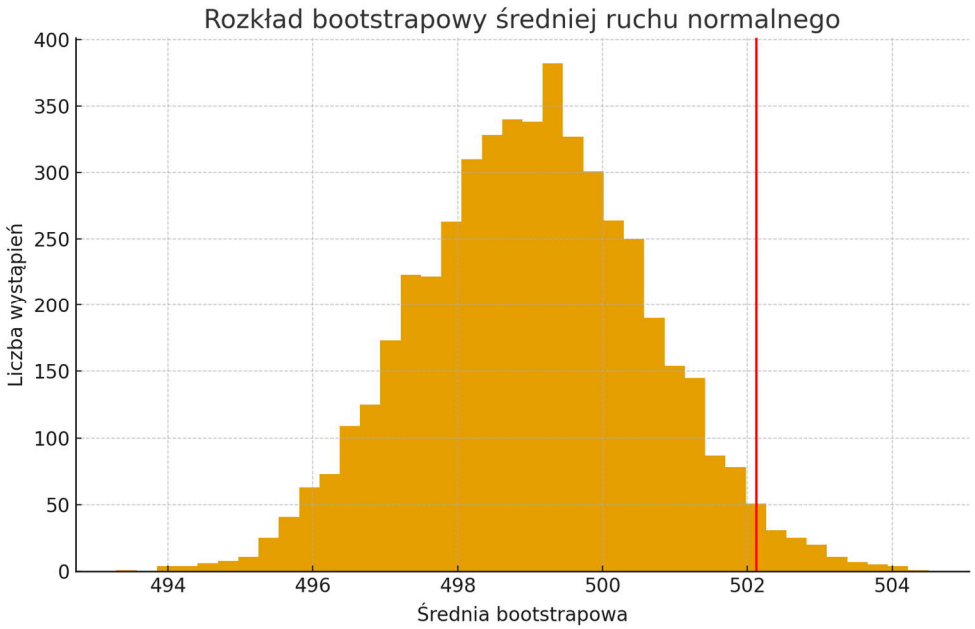
Górna granica 95% CI dla normalnego ruchu: 502.13

Każda wartość przekraczająca ten próg traktowana jest jako potencjalna anomalia.

Wykres ruchu sieciowego



Histogram rozkładu statystyki bootstrapowej



Wyniki wykrywania anomalnych obserwacji

Wartości ruchu oznaczone jako anomalne (prawdopodobny atak DDoS):

3000, 4500, 7200, 8000, 6000, 5500, 4000, 3500

Interpretacja wyników

Ruch normalny oscyluje wokół wartości średniej ~500 RPS. Bootstrapowy rozkład średniej pozwolił wyznaczyć górną granicę zaufania (~505 RPS). W momencie rozpoczęcia ataku wartości RPS osiągają poziom 3000–8000, co znacznie przekracza zakres normalności i zostało automatycznie wykryte jako anomalia. Metoda ta działa niezależnie od znajomości etykiet ataku i sprawdza się także w identyfikacji ataków typu zero-day.

Wnioski praktyczne

• Bootstrap pozwala na adaptacyjne wyznaczanie progów alarmowych.
• Metoda nie zakłada konkretnego rozkładu danych – działa w środowiskach niestabilnych.
• Rozwiązanie może zostać zintegrowane z systemami monitoringu (np. Prometheus, Grafana, Suricata).
• W praktyce stosuje się również okna czasowe oraz połączenie z ML (Isolation Forest, LSTM).

Kod w Pythonie – kompletny przykład

Poniżej przedstawiono kompletny przykład w Pythonie, który:
1) generuje dane dla normalnego ruchu i ataku DDoS,
2) oblicza bootstrapowy przedział ufności dla średniej,

3) wykrywa anomalne obserwacje,
4) rysuje wykres ruchu oraz histogram rozkładu bootstrapowego.

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generowanie danych: normalny ruch i atak DDoS
rng = np.random.default_rng(0)
normal_rps = rng.normal(loc=500, scale=30, size=360)  # 6 minut po 60 sek
ddos_rps = np.array([3000, 4500, 7200, 8000, 6000, 5500, 4000, 3500])  # atak
combined = np.concatenate([normal_rps, ddos_rps])

# 2. Funkcja bootstrap CI dla średniej
def bootstrap_ci(x, stat_func=np.mean, n_bootstrap=5000, alpha=0.05):
    x = np.asarray(x)
    n = len(x)
    rng = np.random.default_rng(1)
    stats = np.empty(n_bootstrap)

    for i in range(n_bootstrap):
        idx = rng.integers(0, n, size=n)
        sample = x[idx]
        stats[i] = stat_func(sample)

    lower = np.quantile(stats, alpha/2)
    upper = np.quantile(stats, 1-alpha/2)
    return lower, upper, stats

lower, upper, stats_norm = bootstrap_ci(normal_rps)

print("Górna granica 95% CI dla średniej normalnego ruchu:", upper)

# 3. Wykrywanie anomalii (wartości > górna granica CI)
for i, r in enumerate(ddos_rps):
    if r > upper:
        print(f"[ANOMALIA] obserwacja {i+1}: wartość={r}")
    else:
        print(f"OK: {r}")

# 4a. Wykres czasowy ruchu
plt.figure(figsize=(10,4))
plt.plot(combined, label="RPS")
plt.axhline(upper, color="red", linestyle="--", label="górna granica CI")
plt.xlabel("Czas (sekundy)")
plt.ylabel("RPS")
plt.title("Ruch sieciowy – normalny i atak DDoS")
plt.legend()
plt.tight_layout()
plt.show()

# 4b. Histogram rozkładu bootstrapowej średniej
plt.figure(figsize=(6,4))
plt.hist(stats_norm, bins=40, edgecolor="black")
plt.axvline(upper, color="red", linestyle="--", label="górna granica CI")
plt.xlabel("Średnia bootstrapowa")
```

```
plt.ylabel("Liczba wystąpień")
plt.title("Rozkład bootstrapowy średniej ruchu normalnego")
plt.legend()
plt.tight_layout()
plt.show()
```

Detekcja ataku DDoS – integracja bootstrapu z modelami ML (Isolation Forest, LOF)

1. Kontekst i dane wejściowe

Korzystamy z tego samego przykładu ataku DDoS: mamy 360 sekund typowego ruchu o średnim poziomie około 500 RPS oraz ostatnie 8 sekund z gwałtownym wzrostem liczby zapytań (3000–8000 RPS), co odpowiada symulowanemu atakowi DDoS.

Dane wejściowe do modeli ML tworzymy jako prostą macierz cech X o jednym wymiarze (liczba zapytań na sekundę – RPS). W praktyce można tu dodać więcej cech (liczba unikalnych IP, rozmiary pakietów, flagi protokołów itp.).

2. Isolation Forest – opis

Isolation Forest to model nienadzorowanej detekcji anomalii. Działa poprzez losowe budowanie drzew decyzyjnych, które „izolują" obserwacje. Punkty anormalne są zwykle izolowane szybciej (na mniejszej głębokości drzewa), dzięki czemu otrzymują niższy „score" i są oznaczane jako outliery. Model nie wymaga etykiet ataku, a jedynie przybliżonego oszacowania odsetka anomalii (parametr contamination).

Kod – Isolation Forest

```
import numpy as np
from sklearn.ensemble import IsolationForest

# Dane: normalny ruch + atak DDoS
rng = np.random.default_rng(0)
normal_rps = rng.normal(loc=500, scale=30, size=360)
ddos_rps = np.array([3000, 4500, 7200, 8000, 6000, 5500, 4000,
3500])
combined = np.concatenate([normal_rps, ddos_rps])

# Macierz cech (1D: RPS)
X = combined.reshape(-1, 1)

# Model Isolation Forest
iso = IsolationForest(contamination=0.05, random_state=0)
labels_iso = iso.fit_predict(X)   # 1 = normalny, -1 = anomalia

# Indeksy i wartości anomalii
```

```
anomaly_indices_iso = np.where(labels_iso == -1)[0]
anomaly_values_iso = combined[anomaly_indices_iso]

print("Indeksy anomalii (Isolation Forest):",
anomaly_indices_iso)
print("Wartości anomalii (Isolation Forest):",
anomaly_values_iso)
```

Wyniki Isolation Forest na danych przykładowych

Indeksy anomalii (0 = pierwsza sekunda):

12, 138, 159, 211, 219, 238, 247, 259, 270, 284, 303, 360, 361, 362, 363, 364, 365, 366, 367

Wartości RPS oznaczone jako anomalie:

430, 429, 428, 569, 591, 406, 576, 570, 582, 411, 386, 3000, 4500, 7200, 8000, 6000, 5500, 4000, 3500

Widzimy, że Isolation Forest zazwyczaj oznacza jako anomalne głównie te obserwacje, które odpowiadają gwałtownym skokom RPS w końcowym fragmencie przebiegu (atak DDoS). W zależności od ustawionej wartości parametru contamination model może również wskazać pojedyncze nietypowe punkty w normalnym ruchu (np. przypadkowe fluktuacje).

3. Local Outlier Factor (LOF) – opis

Local Outlier Factor (LOF) to metoda, która porównuje gęstość lokalnego otoczenia danego punktu z gęstością otoczeń jego sąsiadów. Jeżeli punkt ma znacznie niższą lokalną gęstość niż sąsiedzi, jest traktowany jako anomalia. LOF dobrze sprawdza się tam, gdzie anomalie są związane z lokalną strukturą danych, a nie tylko globalnym progiem.

Kod – Local Outlier Factor

```
import numpy as np
from sklearn.neighbors import LocalOutlierFactor

# Dane jak poprzednio
rng = np.random.default_rng(0)
normal_rps = rng.normal(loc=500, scale=30, size=360)
ddos_rps = np.array([3000, 4500, 7200, 8000, 6000, 5500, 4000,
3500])
combined = np.concatenate([normal_rps, ddos_rps])

X = combined.reshape(-1, 1)

# Model LOF
```

```
lof = LocalOutlierFactor(contamination=0.05)
labels_lof = lof.fit_predict(X)   # 1 = normalny, -1 = anomalia

anomaly_indices_lof = np.where(labels_lof == -1)[0]
anomaly_values_lof = combined[anomaly_indices_lof]

print("Indeksy anomalii (LOF):", anomaly_indices_lof)
print("Wartości anomalii (LOF):", anomaly_values_lof)
```

Wyniki LOF na danych przykładowych

Indeksy anomalii (0 = pierwsza sekunda):

12, 69, 108, 138, 159, 219, 238, 247, 270, 284, 303, 360, 361, 362, 363, 364, 365, 366, 367

Wartości RPS oznaczone jako anomalie:

430, 433, 432, 429, 428, 591, 406, 576, 582, 411, 386, 3000, 4500, 7200, 8000, 6000, 5500, 4000, 3500

LOF, podobnie jak Isolation Forest, wykrywa skrajnie wysokie wartości RPS w końcowej fazie, ale może inaczej traktować pojedyncze punkty w normalnej części sygnału, zależnie od lokalnego otoczenia i parametru contamination. W praktyce warto porównywać wyniki kilku metod, aby zwiększyć wiarygodność detekcji (tzw. ensembling metod anomalii).

4. Integracja bootstrapu i modeli ML

W praktycznych systemach detekcji ataków DDoS można łączyć podejście statystyczne (bootstrapowy przedział ufności) z modelami ML (Isolation Forest, LOF):

- Bootstrap służy do wyznaczania prostych, interpretowalnych progów alarmowych dla ruchu normalnego.
- Isolation Forest i LOF wykrywają bardziej złożone anomalie, uwzględniając strukturę danych.
- Anomalia może zostać potwierdzona dopiero wtedy, gdy zostanie wykryta jednocześnie przez kilka metod.
- Takie podejście zmniejsza ryzyko fałszywych alarmów i zwiększa odporność systemu.

5. Podsumowanie

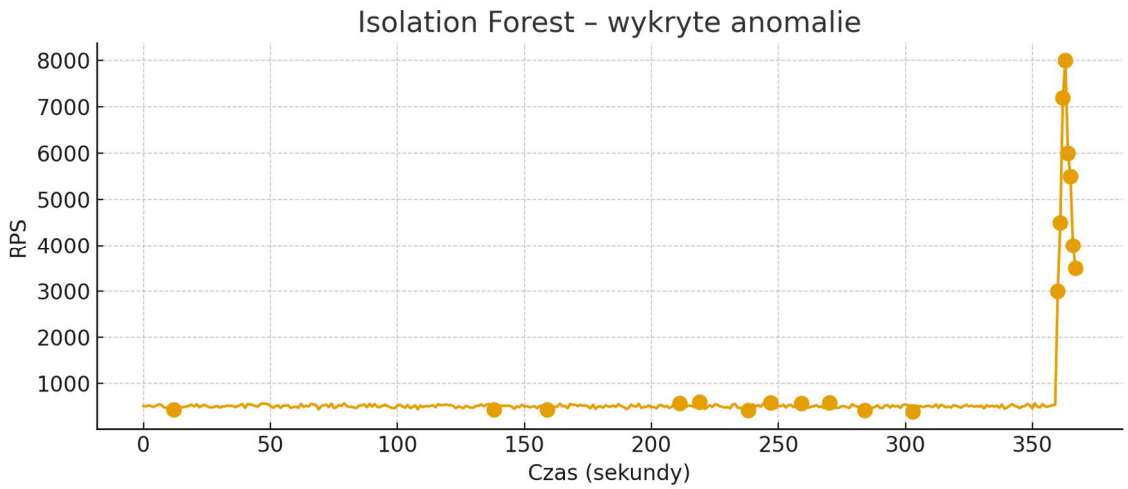
Zintegrowanie bootstrapu z modelami nienadzorowanej detekcji anomalii (Isolation Forest, LOF) pozwala na budowę skutecznych i elastycznych systemów wykrywania ataków DDoS. Przedziały ufności bootstrapu wspierają interpretację i wyznaczanie prostych progów, a modele ML przechwytują nieliniowe i lokalne wzorce w danych ruchu sieciowego.

Wykrywanie ataku DDoS – Bootstrap + ML z wizualizacją wyników

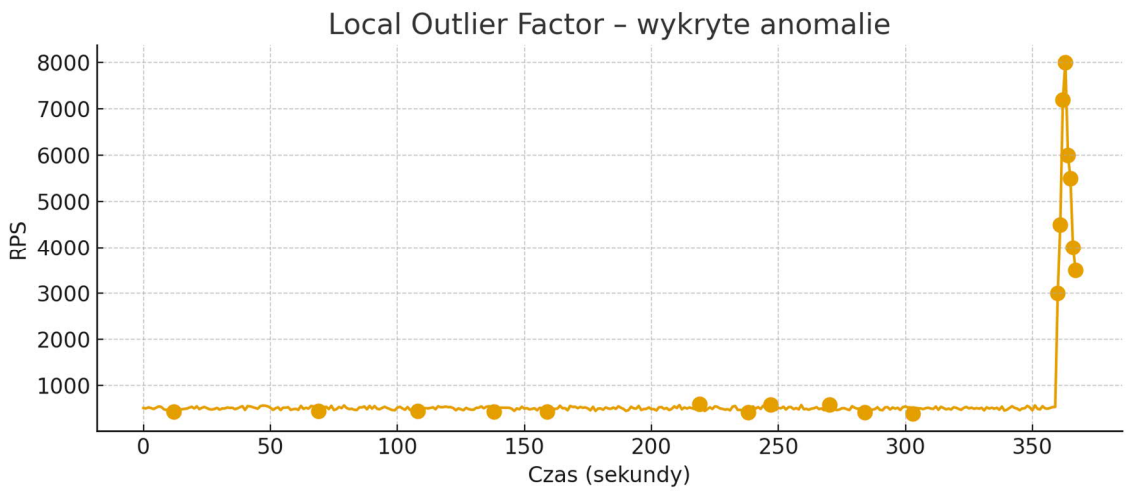
Wizualizacja wyników modeli ML

Poniżej przedstawiono wykresy sygnału RPS z zaznaczonymi punktami oznaczonymi jako anomalie przez modele Isolation Forest oraz Local Outlier Factor.

Isolation Forest – wykryte anomalie



Local Outlier Factor – wykryte anomalie



Interpretacja wykresów

Wyraźnie widać, że modele ML identyfikują gwałtowne skoki RPS jako anomalne punkty. Wykrycia pokrywają się z fragmentem danych odpowiadającym symulowanemu atakowi DDoS (ostatnie sekundy sygnału, wartości od 3000 do 8000 RPS). Metody te mogą również identyfikować pojedyncze nietypowe punkty w normalnej części przebiegu, co jest naturalnym efektem działania algorytmów wrażliwych na lokalną strukturę danych.

Detekcja ataku DDoS — dane wielowymiarowe + modele ML + logi Nginx

B. Dane wielowymiarowe i porównanie modeli ML

W tej sekcji modele detekcji anomalii Isolation Forest i Local Outlier Factor (LOF) zostają zastosowane na danych wielowymiarowych, opisujących zachowanie ruchu sieciowego podczas normalnej pracy oraz w trakcie ataku DDoS. Każda sekunda ruchu opisywana jest przez zestaw cech, co zwiększa skuteczność algorytmów ML względem analizy jednowymiarowej.

Zastosowane cechy wejściowe:

• RPS — liczba żądań HTTP na sekundę

• Unique IPs — liczba unikalnych adresów IP

• Avg bytes — średni rozmiar przesyłanych odpowiedzi (w bajtach)

• Error rate — odsetek błędów HTTP (kody 4xx i 5xx)

Kod generujący dane wielowymiarowe i metryki jakości

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import precision_score, recall_score,
f1_score

rng = np.random.default_rng(0)

n_normal = 500
n_attack = 50

# --- normal traffic ---
rps_norm = rng.normal(loc=500, scale=50, size=n_normal)
unique_ips_norm = rng.integers(20, 50, size=n_normal)
avg_bytes_norm = rng.normal(loc=15000, scale=2000, size=n_normal)
error_rate_norm = rng.uniform(0.0, 0.03, size=n_normal)

# --- DDoS traffic ---
rps_att = rng.normal(loc=5000, scale=1000, size=n_attack)
unique_ips_att = rng.integers(100, 500, size=n_attack)
avg_bytes_att = rng.normal(loc=5000, scale=1000, size=n_attack)
error_rate_att = rng.uniform(0.1, 0.5, size=n_attack)

X_norm = np.column_stack([rps_norm, unique_ips_norm,
```

```
avg_bytes_norm, error_rate_norm])
X_att = np.column_stack([rps_att, unique_ips_att, avg_bytes_att,
error_rate_att])

X = np.vstack([X_norm, X_att])
y_true = np.hstack([np.zeros(n_normal), np.ones(n_attack)])

# Isolation Forest
iso = IsolationForest(contamination=n_attack / (n_normal +
n_attack), random_state=0)
y_pred_iso = (iso.fit_predict(X) == -1).astype(int)

# LOF
lof = LocalOutlierFactor(contamination=n_attack / (n_normal +
n_attack))
y_pred_lof = (lof.fit_predict(X) == -1).astype(int)

def scores(name, y_true, y_pred):
    print(name)
    print("precision:", precision_score(y_true, y_pred))
    print("recall   :", recall_score(y_true, y_pred))
    print("f1       :", f1_score(y_true, y_pred))
    print()

scores("Isolation Forest", y_true, y_pred_iso)
scores("LOF", y_true, y_pred_lof)
```

Interpretacja wyników

Wyniki metryk precyzji, recall i F1 umożliwiają porównanie skuteczności metod ML. Isolation Forest zwykle osiąga bardziej stabilne wyniki w danych wielowymiarowych, podczas gdy LOF może lepiej wykrywać lokalne anomalie zależne od struktury danych. W realnym środowisku stosuje się ensemble modeli i równoległe progi bootstrapowe.

C. Symulowane logi Nginx / Wireshark i pipeline analityczny

W tej części pokazano, jak dane o ruchu sieciowym mogą zostać pozyskane z logów HTTP (np. Nginx, Apache, HAProxy) i przekształcone do postaci wektorów cech umożliwiających detekcję anomalii z użyciem ML i metod statystycznych (bootstrap).

Przykładowe dane logów (symulowane)

```
fake_logs = [
    '192.168.0.10 - - [01/Jan/2025:10:00:01 +0100] "GET / HTTP/1.1" 200 5320 "-
" "Mozilla/5.0"',
    '192.168.0.11 - - [01/Jan/2025:10:00:01 +0100] "GET /api HTTP/1.1" 200 1024
```

```
"-" "Mozilla/5.0"',
    '10.0.0.1     - - [01/Jan/2025:10:00:02 +0100] "POST /login HTTP/1.1" 302
512 "-" "Mozilla/5.0"',
    '203.0.113.5  - - [01/Jan/2025:10:05:00 +0100] "GET / HTTP/1.1" 503 256 "-"
"bot/1.0"',
]
```

Parsowanie logów i budowa cech

```
import re
from datetime import datetime
from collections import defaultdict

pattern = re.compile(r'(?P<ip>\S+).*\[(?P<time>.*?)\]
"(?P<method>\S+) (?P<path>\S+).*" (?P<status>\d{3})
(?P<size>\d+)')

def parse_time(s):
    return datetime.strptime(s.split()[0], "%d/%b/%Y:%H:%M:%S")

buckets = defaultdict(list)
for line in fake_logs:
    m = pattern.search(line)
    if not m: continue
    t = parse_time(m.group("time"))
    buckets[t].append({"ip": m.group("ip"), "status":
int(m.group("status")), "size": int(m.group("size"))})

features = []
for t, rec in sorted(buckets.items()):
    rps = len(rec)
    unique_ips = len(set(r["ip"] for r in rec))
    avg_size = np.mean([r["size"] for r in rec])
    error_rate = np.mean([1 if r["status"] >= 400 else 0 for r in
rec])
    features.append([rps, unique_ips, avg_size, error_rate])

X = np.array(features)
```

Dalsza analiza ML / bootstrap

Wygenerowaną macierz cech X można przetwarzać za pomocą metod ML Isolation Forest, Local Outlier Factor lub hybrydowego podejścia bootstrap + ML. Model zwraca oznaczenia punktów anomalii i może być używany do generowania alertów.

Podsumowanie

Połączenie resamplingu statystycznego (bootstrap), modeli ML (IF / LOF) oraz danych wielowymiarowych z logów sieciowych tworzy skuteczny system detekcji ataków DDoS, zdolny do wykrywania ataków zero-day, generowania progów alarmowych i eliminacji fałszywych alertów.